

Usability Study Report - PaletteForge

Author: Aishwarya Rajmohan

Application description: This study covers Wireframe Studio, the half of PaletteForge that I built. It's a lightweight layout tool: sketch a mockup by adding shapes to a board, drag/resize/recolor them by hand, or pull a color palette straight out of an uploaded image and apply or shuffle it across your shapes. Wireframes and palettes both save to a personal library so you can come back and reload them later. (PaletteForge also includes a separate Accessibility Toolkit built by my partner Priyan Baskar, covering WCAG contrast checking - that half is out of scope for this study; he is running his own usability study on it.)

Video links:

1. https://youtu.be/7eA-bt_c_E : Sandeep, a Full Stack Engineer
2. <https://youtu.be/626xDIQ3FSc> : Surbhi, a Backend Engineer
3. <https://youtu.be/bo-pWKH0Z6A> : Parth, a Hardware Engineer

User Personas:

1. A senior software engineer – full stack
2. A junior software engineer – backend
3. A hardware engineer (non-software background).

None are professional designers and the spread deliberately varies technical/software seniority, to see whether prior software experience (or its absence, for the hardware engineer) affects how intuitively someone navigates the app.

Data description:

Wireframe Studio stores two kinds of data per account: saved wireframes (shape positions, sizes, and colors) and extracted color palettes (pulled from uploaded images, with each color's hex value). Both are private per logged-in user and shown in a personal library the user can reload, edit, or delete from.

Main tasks - use cases:

- T1: Build a mini layout. Add at least two shapes to the wireframe board, move them apart, resize one, and give each a different fill color.
- T2: Get a palette from a picture. Upload any image and get a palette extracted from it, then save that palette.
- T3: Apply and shuffle. Apply the saved palette to your shapes, then use Shuffle at least twice and describe what changes.
- T4: (a) Match a color by hand. Select a shape, then click one of the palette's swatches directly to recolor just that shape. (OR) (b) Palette editing (add/remove/edit colors directly in a saved palette).
- T5: Save and come back. Save your wireframe under a name, start a new one, then reload the one you just saved from your library.

Demographic questions:

- Age range? (e.g., 18–24, 25–34, ...)
- Do you have any design or front-end background? (none / a little / a lot)
- Roughly how often do you use design tools (Figma, Canva, etc.)? (never / rarely / sometimes / often)

Participant 1: Sandeep, A senior software engineer – full stack

Demographics answers

- Age range: 25–34
- Very familiar with web development
- Used design tools often

Detailed notes

- Notes for initial approach: Looked around the app and correctly explained what it was without prompting. Found it very straightforward and intuitive from first glance.
- Notes for T1 (add shapes, add text, recolor): Completed easily. Expected shape text to be directly editable - tried double-clicking the text to edit it in place, which isn't currently supported (text is a fixed placeholder toggled on/off, not an editable field). Suggested this as an improvement. Everything else about the task went smoothly.
- Notes for T2 (extract palette from image, save with a new name): Went well - extracted a palette, edited a color in it, and saved it under a new name without difficulty.
- Notes for T3 (apply palette, shuffle repeatedly, observe): Shuffled many times and correctly observed the color reassignment behavior. No issues.
- Notes for palette editing (add/remove/edit colors directly in a saved palette): Easy, no friction.
- Notes for T5 (save wireframe, start new, reload saved one): Easy - save, new, and reload all worked as expected.

Post-test questionnaire results (Likert scales):

- Score for T1: effectiveness: 4 | intuitiveness: 5
- Score for T2: effectiveness: 5 | intuitiveness: 5
- Score for T3: effectiveness: 5 | intuitiveness: 5
- Score for T4: effectiveness: 5 | intuitiveness: 5
- Score for T5: effectiveness: 5 | intuitiveness: 5
- Overall: effectiveness: 5 | intuitiveness: 5
- Final comments: As a UI designer's tool, very easy to understand - reference an image, get colors out of it, and build a custom palette from there. Very straightforward, no complex wording anywhere.
- Suggested improvements:
 1. support for more shape types,
 2. the saved-wireframes/palettes library gets long - could use a way to collapse it or paginate it.

Participant 2: Surbhi, A junior software engineer – backend

Demographics answers

- Age range: 25–34
- Familiar with web development
- Minimal experience with frontend/design tools

Detailed notes

- Notes for initial approach: Correctly understood the app as a drawing tool and an “image to color converter,” without needing it explained.
- Notes for T1 (create layout, add shapes/colors, resize): Completed the task. Like Sandeep, assumed shape text would be directly editable (double-click to edit) - this was the second participant in a row to expect this.
- Notes for T2 (create palette - add color, remove color, save with name): Completed the save successfully, but there was no visible acknowledgment that the save had actually happened - no confirmation message or indicator, so she wasn't fully sure it worked until checking the saved list herself.
- Notes for T3 (apply palette, shuffle, observe): Went further than asked - on her own, she connected the shuffle feature to visualizing contrast between colors, suggesting the tool's purpose came across clearly.
- Notes for T4 (apply a color to a shape / manual matching): No issues, worked as expected.
- Notes for T5 (save and load): Same issue as T2 - saved and reloaded successfully, but again no acknowledgment message confirming the save.

Post-test questionnaire results (Likert scales):

- Score for T1: effectiveness: 4 | intuitiveness: 5
- Score for T2: effectiveness: 5 | intuitiveness: 4
- Score for T3: effectiveness: 5 | intuitiveness: 5
- Score for T4: effectiveness: 5 | intuitiveness: 5
- Score for T5: effectiveness: 5 | intuitiveness: 4
- Overall: effectiveness: 4 | intuitiveness: 4
- Final comments: Very easy to use app.
- Suggested improvements:
 - Editable text
 - Acknowledgement messages for saving/updating
 - Show recently saved on top so it is obvious to user

Participant 3: Parth, A hardware engineer

Demographics answers

- Age range: 25–34
- No design experience
- Has never used a tool like Canva

Detailed notes

- Notes for initial approach: Correctly identified the layout at a glance (left side for drawing, right side for colors), but was unfamiliar with the native color-picker interface and was generally new to this kind of tool end to end.
- Notes for T1 (create layout, save): Completed the task, but had trouble with the color picker specifically - described the recoloring interaction as feeling like “a paint app” once he figured it out.
- Notes for T2 (create palette, save with name): Completed the save, but didn't understand the percentage numbers shown next to each extracted color or why they were there. Also felt the extracted swatches didn't visually seem to correspond to the uploaded image (no way to see the source image next to the swatches for comparison).
- Notes for T3 (apply palette, shuffle): No issues.
- Notes for T4 (edit palette colors - add/remove/edit): No issues completing the task, but raised that the percentage numbers stop making sense once you've edited a palette's colors, or when reopening an older saved palette.
- Notes for T5 (save and create new): No issues.

Post-test questionnaire results (Likert scales):

- Score for T1: effectiveness: 5 | intuitiveness: 5
- Score for T2: effectiveness: 3 | intuitiveness: 5
- Score for T3: effectiveness: 5 | intuitiveness: 5
- Score for T4: effectiveness: 5 | intuitiveness: 5
- Score for T5: effectiveness: 5 | intuitiveness: 5
- Overall: effectiveness: 5 | intuitiveness: 5
- Final comments:
 1. Would benefit from onboarding - descriptions, tooltips, or a short demo - for a completely new user starting from zero context.
 2. The prevalence percentages on extracted colors are confusing, and stop making sense after a color is edited or when reopening an older saved palette - should be explained (e.g., a tooltip) or replaced with something clearer.
 3. Text on shapes should be directly editable - the same expectation independently raised by both other participants.

Prioritized list of issues and corresponding changes:

Issue: Shape text is not directly editable - all 3 participants (Sandeep, Surbhi, Parth) independently assumed double-clicking the text would let them edit it in place.

Change: Make the shape's text editable in place (e.g., double-click to turn it into an input), instead of a fixed placeholder toggled on/off by a checkbox.

Priority: Must (Must, Could, Should, Would)

Was it implemented? How? Yes. In Shape.jsx, double-clicking the shape's text now switches it into an inline `<input>` (auto-focused); pressing Enter or clicking away commits the change, Escape cancels it. Removed the pointer-events: none on the text span that was blocking any interaction with it. Also fixed a

related bug found while testing: the shape's outer container would grow to fit long wrapped text instead of respecting its set size; added overflow: hidden and min-width/height: 0 in Shape.css so text now wraps and clips within the shape's actual bounds instead of resizing it.

Issue: No confirmation feedback after saving a wireframe or palette - Surbhi wasn't sure her save had worked and had to check the list herself to confirm (verified in code: no toast/alert/success message exists anywhere in the save flow).

Change: Add a brief visual confirmation (e.g., a toast or inline “Saved!” message) after a successful save.

Priority: Must (Must, Could, Should, Would)

Was it implemented? How? Yes. Added a new Toast component (Toast.jsx/Toast.css) shown briefly after a successful save. Wired into both handleSave (wireframes) and handlePaletteSave (palettes) in WireframeStudio.jsx, firing only after the API call actually succeeds. Positioned top-center so it doesn't cover the canvas, and styled with the app's existing success-green colors (--color-pass/--color-pass-bg), so green consistently means "success" across the whole app rather than introducing an unrelated color.

Issue: Saved wireframes/palettes

have no defined sort order - editing an item doesn't bring it to the top, making it hard to find what you just worked on (Surbhi).

Change: Sort both lists by updatedAt descending, same pattern already used in contrastPalettes.js.

Priority: Should

Was it implemented? How? Yes. Added .sort({ updatedAt: -1 }) to the GET / routes in both routes/wireframes.js and routes/extractedPalettes.js.

Issue: Extracted color “prevalence” percentages are confusing and become stale/meaningless after a color is manually edited or when reopening an older saved palette

Change: Either explain what the percentage means (e.g., a tooltip: “how much of the image this color covers”), or stop displaying/clear it once a color has been manually edited.

Priority: Should (Must, Could, Should, Would)

Was it implemented? How? Partially - then reconsidered. Several approaches were thought of to explain the percentage: a hover tooltip, then static always-visible text, then a click-to-reveal info icon with refined wording and a custom SVG. None of them made the number feel worth the added UI weight - the percentage itself just wasn't interesting or useful enough to justify explaining it at all. Instead of continuing to patch the explanation, the percentage display was removed entirely, along with its underlying computation in extractPalette.js. The result is cleaner than any of the incremental fixes: plain color swatches with no number to explain, clarify, or have go stale in the first place.

Issue: First-time users with no design-tool background (Parth) had no in-app explanation of what the app does or how to start.

Change: Add short onboarding hints or tooltips for first-time users (e.g., on the first visit to each tool).

Priority: Could (Must, Could, Should, Would)

Was it implemented? How? Yes. Added short, always-visible hint text in five places instead of a dismissible tour: an app tagline explaining what PaletteForge does, a permanent line explaining how image extraction works, a self-clearing empty-state hint on the wireframe board (disappears once a shape

exists), and short explanations above the "Check accessibility" and "Shuffle" buttons. No dismiss buttons or first-visit tracking needed.

Issue: The saved wireframes/palettes library list has no way to collapse or paginate (Sandeep, anticipating this becoming unwieldy with many saved items).

Change: Add pagination or a collapsible section to the library lists once they exceed a certain length.

Priority: Could (Must, Could, Should, Would)

Was it implemented? How? No

Issue: Only two shape types (rectangle, circle) are available (Sandeep).

Change: Add more shape types (e.g., line, triangle).

Priority: Would (Must, Could, Should, Would)

Was it implemented? How? No